4 Arrays, Pointers, and References

Chapter Objectives

4.1 Arrays of objects

4.2 Using pointers to objects

4.3 The **this** pointer

4.4 Using **new** and **delete**

4.5 More about **new** and **delete**

4.6 References

4.7 Passing references to objects

4.8 Returning references

4.9 Independent references and restrictions

THIS chapter examines several important issues involving arrays of objects and pointers to objects. It concludes with a discussion of one of C++'s most important innovations: the reference. The reference is crucial to many C++ features, so a careful reading is advised.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. When one object is assigned to another, what precisely takes place?

2. Can any troubles or side effects occur when one object is assigned to another? (Give an example.)

3. When an object is passed as an argument to a function, a copy of that object is made. Is the copy's constructor function called? Is its destructor called?

4. By default, objects are passed to functions by value, which means that what occurs to the copy inside the function is not supposed to affect the argument used in the call. Can there be a violation of this principle? If so, give an example.

5. Given the following class, create a function called **make_sum()** their returns an object of type **summation**. Have this function prompt the user for a number and then construct an object having this value and return it to the calling procedure. Demonstrate that the function works.

```
class summation
{
     int num;
     long sum; // summation of num
public:
     void set_sum(int n);
     void show_sum()
     {
          cout << num << " summed is " << sum << "\n";
     }
};

void summation::set_sum(int n)
{
     int i;
     num = n;

     sum = 0;
     for(i=1; i<=n; i++)
          sum += i;
}
```

6. In the preceding question, the function **set_sum()** was not defined in line within the **summation** class declaration. Give a reason why this might be necessary for some compilers.

7. Given the following class, show how to add a friend function called **isneg()** that takes one parameter of type **myclass** and returns true if **num** is negative and false otherwise.

```
class myclass
{
    int num;
public:
    myclass(int x) { num = x; }
};
```

8. Can a friend function be friends with more than one class?

4.1 ARRAYS OF OBJECTS

As has been stated several times, objects are variables and have the same capabilities and attributes as any other type of variable. Therefore, it is perfectly acceptable for objects to be arrayed. The syntax for declaring an array of objects is exactly like that used to declare an array of any other type of variable. Further, arrays of objects are accessed just like arrays of other types of variables.

EXAMPLES

1. Here is an example of an array of objects:

```
#include <iostream>
using namespace std;

class samp
{
    int a;
public:
    void set_a(int n) { a = n; }
    int get_a() { return a; }
};

int main()
{
    samp ob[4];
    int i;

    for(i=0; i<4; i++)
        ob[i].set_a(i);

    for(i=0; i<4; i++)
        cout << ob[i].get_a();

    cout << "\n";

    return 0;
}
```

This program creates a four-element array of objects of type **samp** and then loads each element's **a** with a value between 0 and 3. Notice how member functions are called relative to each array element. The array name, in this case **ob**, is indexed; then the member access operator is applied, followed by the name of the member function to be called.

2. If a class type includes a constructor, an array of objects can be initialized. For example, here **ob** is an initialized array:

```
// Initialize an array.
#include <iostream>
using namespace std;

class samp
{
    int a;
public:
    samp(int n) { a = n; }
    int get_a() { return a; }
};

int main()
{
    samp ob[4] = { -1, -2, -3, -4 };
    int i;

    for(i=0; i<4; i++)
        cout << ob[i].get_a() << ' ';

    cout << "\n";

    return 0;
}
```

This program displays **-1 -2 -3 -4** on the screen. In this example, the values -1 through -4 are passed to the **ob** constructor function.

Actually, the syntax shown in the initialization list is shorthand for this longer form (first shown in Chapter 2):

```
samp ob[4] = { samp(-1), samp(-2),
               samp(-3), samp(-4) };
```

However, the form used in the program is more common (although, as you will see, this form will work only with arrays whose constructors take only one argument).

3. You can also have multidimensional arrays of objeects. For example, here is a program that creates a two-dimensional array of objects and initializes them:

```
// Create a two-dimensional array of objects.
#include <iostream>
using namespace std;

class samp
{
    int a;
public:
    samp(int n) { a = n; }
    int get_a() { return a; }
};
```

```
int main()
{
    samp ob[4][2] = { 1, 2,
                      3, 4,
                      5, 6,
                      7, 8,
                    };
    int i;

    for(i=0; i<4; i++)
    {
        cout << ob[i][0].get_a() << ' ';
        cout << ob[i][1].get_a() << "\n";
    }

    cout << "\n";

    return 0;
}
```

This program displays

```
1 2
3 4
5 6
7 8
```

4. As you know, a constructor can take more than one argument. When initializing an array of objects whose constructor takes more than one argument, you must use the alternative form of initialization mentioned earlier. Let's begin with an example:

```
#include <iostream>
using namespace std;

class samp
{
    int a, b;
public:
    samp(int n, int m) { a = n; b = m; }
    int get_a() { return a; }
    int get_b() { return b; }
};

int main()
{
    samp ob[4][2] = { samp(1, 2), samp(3, 4),
                      samp(5, 6), samp(7, 8),
                      samp(9, 10), samp(11, 12),
                      samp(13, 14), samp(15, 16)
                    };
    int i;
```

```
    for(i=0; i<4; i++)
    {
        cout << ob[i][0].get_a() << ' ';
        cout << ob[i][0].get_b() << "\n";
        cout << ob[i][1].get_a() << ' ';
        cout << ob[i][1].get_b() << "\n";
    }

    cout << "\n";

    return 0;
}
```

In this example, **samp**'s constructor takes two arguments. Here, they array **ob** is declared and initialized in **main()** by using direct calls to **samp**'s constructor. This is necessary because the formal C++ syntax allows only one argument at a time in a comma-separated list. There is no way, for example, to specify two (o more) arguments per entry in the list. Therefore, when you initialize arrays of objects that have constructors that take more than one argument, you must use the "long form" initialization syntax rather than the "shorthand form."

Note: *You can always use the long form of initialization even if the object takes only one argument. It's just that the short form is more convenient in this case.*

The preceding program displays

```
1 2
3 4
5 6
7 8
9 10
11 12
13 14
15 16
```

EXERCISES

1. Using the following class declaration, create a ten-element array and initialize the **ch** element with the values A through J. Demonstrate that the array does, indeed, contain these values.

```
#include <iostream>
using namespace std;

class letters
{
    char ch;
public:
    letters(char c) { ch = c; }
    char get_ch() { return ch; }
};
```

2. Using the following class declaration, create a ten-element array, initialize **num** to the values 1 through 10, and initialize **sqr** to **num**'s square.

```
#include <iostream>
using namespace std;

class squares
{
    int num, sqr;
public:
    squares(int a, int b) { num = a; sqr = b; }
    void show() { cout << num << ' ' << sqr << "\n"; }
};
```

3. Change the initialization in Exercise 1 so it uses the long form. (That is, invoke **letters**' constructor explicitly in the initialization list.)

4.2 USING POINTERS TO OBJECTS

As discussed in Chapter 2, objects can be accessed via pointers. As you know, when a pointer to an object is used, the object's members are referenced using the arrow (->) operator instead of the dot (.) operator.
Pointer arithmetic using an object pointer is the same as it is for any other data type: it is performed relative to the type of the object. For example, when an object pointer is incremented, it points to the next object. When an object pointer is decremented, it points to the previous object.

EXAMPLES

1. Here is an example of object pointer arithmetic:

```
// Pointers to objects.
#include <iostream>
using namespace std;

class samp
{
    int a, b;
public:
    samp(int n, int m) { a = n; b = m; }
    int get_a() { return a; }
    int get_b() { return b; }
};

int main()
{
    samp ob[4] = { samp(1, 2),
                   samp(3, 4),
                   samp(5, 6),
                   samp(7, 8)
                 };
    int i;
```

```
    samp *p;
    p = ob; // get starting address of array

    for(i=0; i<4; i++)
    {
        cout << p->get_a() << ' ';
        cout << p->get_b() << "\n";
        p++; // advance to next object
    }

    cout << "\n";

    return 0;
}
```

This program displays

```
1 2
3 4
5 6
7 8
```

As evidenced by the output, each time **p** is incremented, it points to the next object in the array.

EXERCISES

1. Rewrite Example 1 so it displays the contents of the **ob** array in reverse order.
2. Change Section 4.1, Example 3 so the two-dimensional array is accessed via a pointer. Hint: In C++, as in C, all arrays are stored contiguously, left to right, low to high.

4.3 THE this POINTER

C++ contains a special pointer that is called **this**. **this** is a pointer that is automatically passed to any member function when it is called, and it is a pointer to the object that generates the call. For example, given this statement,

```
ob.f1(); // assume that ob is an object
```

the function **f1()** is automatically passed a pointer to **ob**-which is the object that invokes the call. This pointer is referred to as **this**.
It is important to understand that on;y member functions are passed a **this** pointer. For example, a friend does not have a **this** pointer.

EXAMPLE

1. As you have seen, when a member function refers to another member of a class, it does so directly without qualifying the member with either a class or an object specification. For example, examine this short program, which creates a simple inventory class:

```
// Demonstrate the this pointer.
#include <iostream>
#include <cstring>
using namespace std;

class inventory
{
    char item[20];
    double cost;
    int on_hand;
public:
    inventory(char *i, double c, int o)
    {
        strcpy(item, i);
        cost = c;
        on_hand = o;
    }
    void show();
};

void inventory::show()
{
    cout << item;
    cout << ": $" << cost;
    cout << "  On hand: " << on_hand << "\n";
}

int main()
{
    inventory ob("wrench", 4.95, 4);

    ob.show();

    return 0;
}
```

As you can see, within the constructor **inventory()** and the member function **show()**, the member variables **item**, **cost**, and **on_hand** are referred to directly. This is because a member function can be called only in conjunction with an object. Therefore, the compiler knows which object's data is being referred to.

However, there is an even more subtle explanation. When a member function is called, it is automatically passed a **this** pointer to the object that invoked the call. Thus, the preceding program could be rewritten as shown here:

```
// Demonstrate the this pointer.
#include <iostream>
#include <cstring>
using namespace std;

class inventory
{
    char item[20];
```

```
    double cost;
    int on_hand;
public:
    inventory(char *i, double c, int o)
    {
        strcpy(this->item, i); // access members
        this->cost = c; // through the this
        this->on_hand = o; // pointer
    }
    void show();
};

void inventory::show()
{
    cout << this->item; // use this to access members
    cout << ": $" << this->cost;
    cout << "  On hand: " << this->on_hand << "\n";
}

int main()
{
    inventory ob("wrench", 4.95, 4);

    ob.show();

    return 0;
}
```

Here the member variables are accessed explicitly through the **this** pointer. Thus, within **show()**, these two statements are equivalent:

```
cost = 123.23;
this->cost = 123.23;
```

In fact, the first form is, loosely speaking, a shorthand for the second.

While no C++ programmer would use the **this** pointer to access a class member as just shown, because the shorthand form is much easier, it is important to understand what the shorthand implies.

The **this** pointer has several uses, including aiding in overloading operators. This use will be detailed in Chapter 6. For now, the important thing to understand is that by default, all member functions are automatically passed a pointer to the invoking object.

EXERCISE

1. Given the following program, convert all appropriate references to class members to explicit **this** pointer references.

```
#include <iostream>
using namespace std;

class myclass
{
    int a, b;
```

```
public:
    myclass(int n, int m) { a = n; b = m; }
    int add() { return a+b; }
    void show();
};

void myclass::show()
{
    int t;

    t = add(); // call member function
    cout << t << "\n";
}

int main()
{
    myclass ob(10, 14);

    ob.show();

    return 0;
}
```

4.4 USING new AND delete

Up to now, when memory needed to be allocated, you have been using **malloc()**, and you have been freeing allocated memory by using **free()**. These are, of course, the standard C dynamic allocation functions. While these functions are available in C++, C++ provides a safer and more convenient way to allocate and free memory. In C++, you can allocate memory using **new** and release it using **delete**. These operators take these general forms:

```
p_var = new type;
delete p_var;
```

Here *type* is the type specifier of the object for which you want to allocate memory and *p-var* is a pointer to that type. **new** is an operator that returns a pointer to dynamically allocated memory that is large enough to hold an object of type *type*. **delete** releases that memory when it is no longer needed. **delete** can be called only with an invalid pointer, the allocation system will be destroyed, possibly crashing your program.
If there is insufficient available memory to fill an allocation request, one of two actions will occur. Either **new** will return a null pointer or it will generate an exception. (Exceptions and exception handling are described later in this book; loosely, an exception is a run-time error that can be managed in a structured fashion.) In Standard C++, the default behavior of **new** is to generate an exception when it cannot satisfy an allocation request. If this exception is not handled by your program, your program will be terminated. The trouble is that the precise action that **new** takes on failure has been changed several times over the past few years. Thus, it is possible that your compiler will not implement **new** as defined by Standard C++.
When C++ was first invented, **new** returned null on failure. Later this was changed such that **new** caused an exception on failure. Finally, it was decided that a **new** failure will generate an exception by default, but that a null pointer could be returned instead, as an option. Thus, **new** has been implemented differently at different times by compiler manufacturers. For example, at the time of this writing, Microsoft's Visual C++ returns a null pointer when **new** fails.

Borland C++ generates an exception. Although all compilers will eventually implement **new** in compliance with Standard C++, currently the only way to know the precise action of **new** on failure is to check your compiler's documentation.

Since there are two possible ways that **new** can indicate allocation failure, and since different compilers might do so differently, the code in this book will be written in such a way that both contingencies are accommodated. All code in this book will test the pointer returned by **new** for null. This handles compilers that implement **new** by returning null on failure, while causing no harm for those compilers for which **new** throws an exception. If your compiler generates an exception when **new** fails, the program will simply be terminated. Later, when exception handling is described, **new** will be re-examined and you will learn how to better handle an allocation failure. You will also learn about an alternative form of **new** that always returns a null pointer when an error occurs.

One last point: none of the examples in this book should cause **new** to fail, since only a handful of bytes are being allocated by any single program.

Although **new** and **delete** perform functions similar to **malloc()** and **free()**, they have several advantages. First, **new** automatically allocates enough memory to hold an object of the specified type. You do not need to use **sizeof**, for example, to compute the number of bytes required. This reduces the possibility for error. Second, **new** automatically returns a pointer of the specified type. You do not need to use an explicit type cast the way you did when you allocated memory by using **malloc()** (see the following note). Third, both **new** and **delete** can be overloaded, enabling you to easily implement your own custom allocation system. Fourth, it is possible to initialize a dynamically allocated object. Finally, you no longer need to include **<cstdlib>** with your programs.

Note: *In C, no type cast is required when you are assigning the return value of* ***malloc()*** *to a pointer because the* ***void **** *returned by* ***malloc()*** *is automatically converted into a pointer compatible with the type of pointer on the left side of the assignment. However, this is not the case in C++, which requires an explicit type cast when you use* ***malloc()****. The reason for this difference is that it allows C++ to enforce more rigorous type checking.*

Now that **new** and **delete** have been introduced, they will be used instead of **malloc()** and **free()**.

EXAMPLES

1. As a short first example, this program allocates memory to hold an integer:

```
// A simple example of new and delete.
#include <iostream>
using namespace std;

int main()
{
    int *p;

    p = new int; // allocate room for an integer
    if(!p)
    {
        cout << "Allocation error\n";
        return 1;
    }

    *p = 100;
```

```
    cout << "Here is integer at p: " << *p << "\n";

    delete p; // release memory

    return 0;
}
```

Notice that the value returned by **new** is checked before it is used. As explained earlier, this check is meaningful only if your compiler implements **new** is such a way that it returns null on failure.

2. Here is an example that allocates an object dynamically.

```
// Allocating dynamic objects.
#include <iostream>
using namespace std;

class samp
{
    int i, j;
public:
    void set_i(int a, int b) { i=a; j=b; }
    int get_product() { return i*j; }
};

int main()
{
    samp *p;

    p = new samp; // allocate object
    if(!p)
    {
        cout << "Allocation error\n";
        return 1;
    }

    p->set_i(4, 5);

    cout << "Product is: " << p->get_product() << "\n";

    return 0;
}
```

EXERCISES

1. Write a program that uses **new** to dynamically allocate a **float**, a **long**, and a **char**. Give these dynamic variables values and display their values. Finally, release all dynamically allocated memory by using **delete**.

2. Create a class that contains a person's name and telephone number. Using **new**, dynamically allocate an object of this class and put your name and phone number into these fields within this object.

3. What are the two ways that **new** might indicate an allocation failure?

4.5 MORE ABOUT new AND delete

This section discusses two additional features of **new** and **delete**. First, dynamically allocated objects can be given initial values. Second, dynamically allocated arrays can be created.
You can give a dynamically allocated object an initial value by using this form of the **new** statement:

```
p_var = new type (initial_value);
```

To dynamically allocate a one-dimensional array, use this form of **new**:

```
p_var = new type [size];
```

After this statement has executed, *p-var* will point to the start of an array of *size* elements of the type specified. For various technical reasons, it is not possible to initialize an array that is dynamically allocated.
To delete a dynamically allocated array, use this form of **delete**:

```
delete [] p_var;
```

This syntax causes the compiler to call the destructor function for each element in the array. It does *not* cause *p-var* to be freed multiple times. *p-var* is still freed only once.
Note: *For older compilers, you might need to specify the size of the array that you are deleting between the square brackets of the* ***delete*** *statement. This was required by the original definition of C++. However, the size specification is not needed by modern compilers.*

EXAMPLES

1. This program allocates memory for an integer and initializes that memory:

```
// An example of initializing a dynamic variable.
#include <iostream>
using namespace std;

int main()
{
     int *p;

     p = new int(9); // give initial value of 9

     if(!p)
     {
          cout << "Allocation error\n";
          return 1;
     }

     cout << "Here is integer at p: " << *p << "\n";

     delete p; // release memory

     return 0;
}
```

   As you should expect, this program displays the value 9, which is the initial value given to the memory pointed to by **p**.

2. The following program passes initial values to a dynamically allocated object:

```
// Allocating dynamic objects
#include <iostream>
using namespace std;

class samp
{
    int i, j;
public:
    samp(int a, int b) { i=a; j=b; }
    int get_product() { return i*j; }
};

int main()
{
    samp *p;

    p = new samp(6, 5); // allocate object with
       initialization
    if(!p)
    {
        cout << "Allocation error\n";
        return 1;
    }

    cout << "Product is: " << p->get_product() << "\n";

    delete p;

    return 0;
}
```

When the **samp** object is allocated, its constructor is automatically called and is passed the values 6 and 5.

3. The following program allocates an array of integers:

```
// A simple example of new and delete
#include <iostream>
using namespace std;

int main()
{
    int *p;

    p = new int [5]; // allocate room for 5 integers

    // always make sure that allocation succeeded
    if(!p)
    {
        cout << "Allocation error\n";
        return 1;
    }
```

```
    int i;

    for(i=0; i<5; i++)
        p[i] = i;

    for(i=0; i<5; i++)
    {
        cout << "Here is integer at p[" << i << "]: ";
        cout << p[i] << "\n";
    }

    delete [] p;

    return 0;
}
```

This program displays the following:

```
Here is integer at p[0]:  0
Here is integer at p[1]:  1
Here is integer at p[2]:  2
Here is integer at p[3]:  3
Here is integer at p[4]:  4
```

4. The following program creates a dynamic array of objects:

```
// Allocating dynamic objects.
#include <iostream>
using namespace std;

class samp
{
    int i, j;
public:
    void set_ij(int a, int b) { i=a; j=b; }
    int get_product() { return i*j; }
};

int main()
{
    samp *p;
    int i;

    p = new samp [10]; // allocate object array
    if(!p)
    {
        cout << "Allocation error\n";
        return 1;
    }

    for(i=0; i<10; i++)
```

```
        p[i].set_ij(i, i);

    for(i=0; i<10; i++)
    {
        cout << "Product [" << i << "] is: ";
        cout << p[i].get_product() << "\n";
    }
    delete [] p;

    return 0;
}
```

This program displays the following:

```
Here is integer at p[0]:  0
Product [0] is:  0
Product [1] is:  1
Product [2] is:  4
Product [3] is:  9
Product [4] is:  16
Product [5] is:  25
Product [6] is:  36
Product [7] is:  49
Product [8] is:  64
Product [9] is:  81
```

5. The following version of the preceding program gives **samp** a destructor, and now when **p** is freed, each element's destructor is called:

```
// Allocating dynamic objects
#include <iostream>
using namespace std;

class samp
{
    int i, j;
public:
    void set_ij(int a, int b) { i=a; j=b; }
    ~samp() { cout << "Destroying...\n"; }
    int get_product() { return i*j; }
};

int main()
{
    samp *p;
    int i;

    p = new samp [10]; // allocate object array
```

```
    if(!p)
    {
        cout << "Allocation error\n";
        return 1;
    }

    for(i=0; i<10; i++)
        p[i].set_ij(i, i);

    for(i=0; i<10; i++)
    {
        cout << "Product [" << i << "] is: ";
        cout << p[i].get_product() << "\n";
    }
    delete [] p;

    return 0;
}
```

This program displays the following:

```
Here is integer at p[0]:  0
Product [0] is:  0
Product [1] is:  1
Product [2] is:  4
Product [3] is:  9
Product [4] is:  16
Product [5] is:  25
Product [6] is:  36
Product [7] is:  49
Product [8] is:  64
Product [9] is:  81
Destroying...
Destroying...
Destroying...
Destroying...
Destroying...
Destroying...
Destroying...
Destroying...
Destroying...
Destroying...
```

As you can see, **samp**'s destructor is called ten times-once for each element in the array.

EXERCISES

1. Show how to convert the following code into its equivalent that uses **new**.

```
char *p;

p = (char *) malloc(100);
// ...
strcpy(p, "This is a test");
```

   Hint: A string is simply an array of characters.

2. Using **new**, show how to allocate a **double** and give it an initial value of -123.0987.

4.6 REFERENCES

C++ contains a feature that is related to the pointer: the *reference*. A reference is an implicit pointer that for all intents and purposes acts like another name for a variable. There are three ways that a reference can be used. First, a reference can be passed to a function. Second, a reference can be returned by a function. Finally, an independent reference can be created. Each of these applications of the reference is examined, beginning with reference parameters.
Without a doubt, the most important use of a reference is as a parameter to a function. To help you understand what a reference parameter is and how it works, let's first start with a program that uses a pointer (not a reference) as a parameter:

```
#include <iostream>
using namespace std;

void f(int *n); // use a pointer parameter

int main()
{
    int i = 0;

    f(&i);

    cout << "Here is i's new value: " << i << "\n";

    return 0;
}

void f(int *n)
{
    *n = 100; // put 100 into the argument pointed to by n
}
```

Here **f()** loads the value 100 into the integer pointed to by **n**. In this program, **f()** is called with the address of **i** in **main()**. Thus, after **f()** returns, **i** contains the value 100.
This program demonstrates how a pointer is used as a parameter to manually create a call-by-reference parameter-passing mechanism. In a C program, this is the only way to achieve a call-by-reference.

However, in C++, you can completely automate this process by using a reference parameter. To see how, let's rework the previous program. Here is a version that uses a reference parameter:

```
#include <iostream>
using namespace std;

void f(int &n); // declare a reference parameter

int main()
{
    int i = 0;

    f(i);

    cout << "Here is i's new value: " << i << "\n";

    return 0;
}

void f(int &n)
{
    // notice that no * is needed in the following statement
    n = 100; // put 100 into the argument used to call f()
}
```

Examine this program carefully. First, to declare a reference variable or parameter, you precede the variable's name with the **&**. This is how **n** is declared as a parameter to **f()**. Now that **n** is a reference, it is no longer necessary-or even legal-to apply the * operator. Instead, each time **n** is used within **f()**, it is automatically treated as a pointer to the argument used to call **f()**. This means that the statement

```
n = 100;
```

actually puts the value 100 into the variable used to call **f()**, which in this case, is **i**. Further, when **f()** is called, there is no need to precede the argument with the **&**. Instead, because **f()** is declared as taking a reference parameter, the address to the argument is *automatically* passed to **f()**.

To review, when you use a reference parameter, the compiler automatically passes the address of the variable used as the argument. There is no need to manually generate the address of the argument by preceding it with an **&** (in fact, it is not allowed). Further, within the function, the compiler automatically uses the variable pointed to by the reference parameter. There is no need to employ the * (and again, it is not allowed). Thus, a reference parameter fully automates the call-by-reference parameter-passing mechanism.

It is important to understand that you cannot change what a reference is pointing to. For example, if the statement

```
n++;
```

were put inside **f()** in the preceding program, **n** would still be pointing to **i** in **main()**. Instead of incrementing **n**, this statement increments the value of the variable being referenced (in this case, **i**).

Reference parameters offer several advantages over their (more or less) equivalent pointer alternatives. First, from a practical point of view, you no longer need to remember to pass the address of an argument. When a reference parameter is used, the address is automatically passed. Second, in the opinion of many programmers, reference parameters offer a cleaner,

more elegant interface than the rather clumsy explicit pointer mechanism. Third, as you will see in the next section, when an object is passed to a function as a reference, no copy is made. This is one way to eliminate the troubles associated with the copy of an argument damaging something needed elsewhere in the program when its destructor function is called.

EXAMPLES

1. The classic example of passing arguments by reference is a function that exchanges the values of the two arguments with which it is called. Here is an example called **swapargs()** that uses references to swap its two integer arguments:

```
#include <iostream>
using namespace std;

void swapargs(int &x, int &y);

int main()
{
    int i, j;

    i = 10;
    j = 19;

    cout << "i: " << i << ", ";
    cout << "j: " << j << "\n";

    swapargs(i, j);

    cout << "After swapping: ";

    cout << "i: " << i << ", ";
    cout << "j: " << j << "\n";

    return 0;
}

void swapargs(int &x, int &y)
{
    int t;

    t = x;
    x = y;
    y = t;
}
```

If **swapargs()** had been written using pointers instead of references, it would have looked like this:

```
void swapargs(int *x, int *y)
{
    int t;

    t = *x;
```

```
    *x = *y;
    *y = t;
}
```

As you can see, by using the reference version of **swapargs()**, the need for the * operator is eliminated.

2. Here is a program that uses the **round()** function to round a **double** value. The value to be rounded is passed by reference.

```
#include <iostream>
#include <cmath>
using namespace std;

void round(double &num);

int main()
{
    double i = 100.4;

    cout << i << " rounded is ";
    round(i);
    cout << i << "\n";

    i = 10.9;
    cout << i << " rounded is ";
    round(i);
    cout << i << "\n";

    return 0;
}

void round(double &num)
{
    double frac;
    double val;

    // decompose num into whole and fractional parts
    frac = modf(num, &val);

    if(frac < 0.5)
        num =val;
    else
        num = val+1.0;
}
```

round() uses a relatively obscure standard library function called **modf()** to decompose a number into its whole number and fractional parts. The fractional part is returned; the whole number is put into the variable pointed to by **modf()**'s second parameter.

EXERCISES

1. Write a function called **neg()** that reverses the sign of its integer parameter. Write the function two ways-first by using a pointer parameter and then by using a reference parameter. Include a short program to demonstrate their operation.

2. What is wrong with the following program?

```
// This program has an error.
#include <iostream>
using namespace std;

void triple(double &num);

int main()
{
    double d = 7.0;

    triple(&d);

    cout << d;
    return 0;
}

// Triple num's value.
void triple(double &num)
{
    num = 3 * num;
}
```

3. Give some advantages of reference parameters.

4.7 PASSING REFERENCES TO OBJECTS

As you learned in Chapter 3, when an object is passed to a function by use of the default call-by-value parameter-passing mechanism, a copy of that object is made. Although the parameter's constructor function is not called, its destructor function is called when the function returns. As you should recall, this can cause serious problems in some instances-when the destructor frees dynamic memory, for example.

One solution to this problem is to pass an object by reference. (The other solution involves the use of copy constructors, which are discussed in Chapter 5.) When you pass the object by reference, no copy is made, and therefore its destructor function is not called when the function returns. Remember, however, that changes made to the object inside the function affect the object used as the argument.

Note: *It is critical to understand that a reference is not a pointer. Therefore, when an object is passed by reference, the member access operator remains the dot (.), not the arrow (->)*

EXAMPLES

1. The following is an example that demonstrates the usefulness of passing an object by reference. First, here is a version of a program that passes an object of **myclass** by value to a function called **f()**:

```
#include <iostream>
using namespace std;
```

```
class myclass
{
    int who;
public:
    myclass(int n)
    {
        who = n;
        cout << "Constructing " << who << "\n";
    }
    ~myclass() { cout << "Destructing " << who << "\n"; }
    int id() { return who; }
};

// o is passed by value.
void f(myclass o)
{
    cout << "Received " << o.id() << "\n";
}

int main()
{
    myclass x(1);

    f(x);

    return 0;
}
```

This function displays the following:

```
Constructing 1
Received 1
Destructing 1
Destructing 1
```

As you can see, the destructor function is called twice-first when the copy of object 1 is destroyed when **f()** terminates and again when the program finishes.

However, if the program is changed so that **f()** uses a reference parameter, no copy is made and, therefore, no destructor is called when **f()** returns:

```
#include <iostream>
using namespace std;

class myclass
{
    int who;
public:
    myclass(int n)
    {
        who = n;
        cout << "Constructing " << who << "\n";
```

```
    }
    ~myclass() { cout << "Destructing " << who << "\n"; }
    int id() { return who; }
};

// Now o is passed by reference.
void f(myclass &o)
{
    // note that . operator is still used!!
    cout << "Received " << o.id() << "\n";
}

int main()
{
    myclass x(1);

    f(x);

    return 0;
}
```

This version displays the following output:

```
Constructing 1
Received 1
Destructing 1
```

Remember: *When accessing members of an object by using a reference, use the dot operator, not the arrow.*

EXERCISE

1. What is wrong with the following program? Show how it can be fixed by using a reference parameter.

```
// This program has an error.
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
public:
    strtype(char *s);
    ~strtype() { delete [] p; }
    char *get() { return p; }
};

strtype::strtype(char *s)
{
```

```
    int l;

    l = strlen(s)+1;

    p = new char [l];
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }

    strcpy(p, s);
}

void show(strtype x)
{
    char *s;

    s = x.get();
    cout << s << "\n";
}

int main()
{
    strtype a("Hello"), b("There");

    show(a);
    show(b);

    return 0;
}
```

4.8 RETURNING REFERENCES

A function can return a reference. As you will see in Chapter 6, returning a reference can be very useful when you are overloading certain types of operators. However, it also can be employed to allow a function to be used on the left side of an assignment statement. The effect of this is both powerful and startling.

EXAMPLES

1. To begin, here is a very simple program that contains a function that returns a reference:

```
// A simple example of a function returning a reference.
#include <iostream>
using namespace std;

int &f(); // return a reference
int x;

int main()
```

```
{
     f() = 100; // assign 100 to reference returned by f()

     cout << x << "\n";

     return 0;
}

// Return an int reference
int &f()
{
     return x; // returns a reference to x
}
```

Here function **f()** is declared as returning a reference to an integer. Inside the body of the function, the statement

```
return x;
```

does *not* return the value of the global variable **x**, but rather, it automatically returns **x**'s address (int the form of a reference). Thus, inside **main()**, the statement

```
f() = 100;
```

puts the value 100 into **x** because **f()** has returned a reference to it.

To review, function **f()** returns a reference. Thus when **f()** is used on the left side of the assignment statement, it is this reference, returned by **f()**, that is being assigned to. Since **f()** returns a reference to **x** (in this example), it is **x** that receives the value 100.

2. You must be careful when returning a reference that the object you refer to does not go out of scope. For example, consider this slight reworking of function **f()**:

```
// Return an int reference.
int &f()
{
     int x; // x is now a local variable
     return x; // returns a reference to x
}
```

In this case, **x** is now local to **f()** and will go out of scope when **f()** returns. This effectively means that the reference returned by **f()** is useless.

Note: *Some C++ compilers will not allow you to return a reference to a local variable. However, this type of problem can manifest itself in other ways, such as when objects are allocated dynamically.*

3. One very good use of returning a reference is found when a bounded array type is created. As you know, in C and C++, no array boundary checking occurs. It is therefore possible to overflow or underflow an array. However, in C++, you can create an array class that performs automatic bounds checking. An array class contains two core functions-one that stores information into the array and one that retrieves information. These functions can check, at run time, that the array boundaries are not overrun.

   The following program implements a bounds-checking array for characters:

```
// A simple bounded array example.
#include <iostream>
#include <cstdlib>
using namespace std;

class array
{
    int size;
    char *p;
public:
    array(int num);
    ~array() { delete [] p; }
    char &put(int i);
    char get(int i);
};

array::array(int num)
{
    p = new char [num];
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    size = num;
}

// Put something into the array.
char &array::put(int i)
{
    if(i<0 || i>=size)
    {
        cout << "Bounds error!!!\n";
        exit(1);
    }
    return p[i]; // return reference to p[i]
}

// Get something from the array.
char array::get(int i)
{
    if(i<0 || i>=size)
    {
        cout << "Bounds error!!!\n";
        exit(1);
    }
    return p[i];
}

int main()
{
```

```
    array a(10);
    a.put(3) = 'X';
    a.put(2) = 'R';

    cout << a.get(3) << a.get(2);
    cout << "\n";

    // now generate run-time boundary error
    a.put(11) = '!';

    return 0;
}
```

This example is a practical use of functions returning references, and you should examine it closely. Notice that the **put()** function returns a reference to the array element specified by parameter **i**. This reference can them be used on the left side of an assignment statement to store something in the array-if the index specified by **i** is not out of bounds. The reverse is **get()**, which returns the value stored at the specified index if that index is within range. This approach to maintaining an array is sometimes referred to as a *safe array*. (You will see a better way to create a safe array later on, in Chapter 6.)

One other thing to notice about the preceding program is that the array is allocated dynamically by the use of **new**. This allows arrays of differing length to be declared.

As mentioned, the way that bounds checking is performed in this program is a practical application of C++. If you need to have array boundaries verified at run time, this is one way to do it. However, remember that bounds checking slows access to the array. Therefore, it is best to include bounds checking only when there is a real likelihood that an array boundary will be violated.

EXERCISES

1. Write a program that creates a two-by-three two-dimensional safe array of integers. Demonstrate that it works.

2. Is the following fragment valid? If not, why not?

```
int &f();
.
.
.
int *x;

x = f();
```

4.9 INDEPENDENT REFERENCES AND RESTRICTIONS

Although not commonly used, the *independent reference* is another type of reference that is available in C++. An independent reference is a reference variable that in all effects is simply another name for another variable. Because references cannot be assigned new values, an independent reference must be initialized when it is declared.

Note: *Because independent references are sometimes used, it is important that you know about them. However, most programmers feel that there is no need for them and that they can add*

confusion to a program. Further, independent references exist in C++ largely because there was no compelling reason to disallow them. But for the most part, their use should be avoided.
There are a number of restrictions that apply to all types of references. You cannot reference another reference. You cannot obtain the address of a reference. You cannot create arrays of references, and you cannot reference a bit-field. References must be initialized unless they are members of a class, are return values, or are function parameters.
Remember: *References are similar to pointers, but they are not pointers.*

EXAMPLES

1. Here is a program that contains an independent reference:

```
#include <iostream>
using namespace std;

int main()
{
    int x;
    int &ref = x; // create an independent reference

    x = 10; // these two statements
    ref = 10; // are functionally equivalent

    ref = 100;
    // this prints the number 100 twice
    cout << x << ' ' << ref << "\n";

    return 0;
}
```

   In this program, the independent reference **ref** serves as a different name for **x**. From a practical point of view, **x** and **ref** are equivalent.

2. An independent reference can refer to a constant. For example, this is valid:

```
const int &ref = 10;
```

   Again, there is little benefit in this type of reference, but you may see it from time too time in other programs.

EXERCISE

1. On your own, try to think of a good use for an independent reference.

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. Given the following class, create a two-by-five two-dimensional array and give each object in the array an initial value of your own choosing. Then display the contents of the array.

```
class a_type
{
    double a, b;
public:
    a_type(double x, double y)
    {
        a = x;
        b = y;
    }
    void show() { cout << a << ' ' << b << "\n"; }
};
```

2. Modify your solution to the preceding problem so it accesses the array by using a pointer.

3. What is the **this** pointer?

4. Show the general forms for **new** and **delete**. What are some advantages of using them instead of **malloc()** and **free()**?

5. What is a reference? What is one advantage of using a reference parameter?

6. Create a function called **recip()** that takes one **double** reference parameter. Have the function change the value of that parameter into its reciprocal. Write a program to demonstrate that it works.

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapters.

1. Given a pointer to an object, what operator is used to access a member of that object?

2. In Chapter 2, a **strtype** class was created that dynamically allocated space for a string. Rework the **strtype** class (shown here for your convenience) so it uses **new** and **delete**.

```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *ptr);
    ~strtype();
    void show();
};

strtype::strtype(char *ptr)
{
    len = strlen(ptr);
    p = (char *) malloc(len+1);
```

```
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    strcpy(p, ptr);
}

strtype::~strtype()
{
   cout << "Freeing p\n";
   free(p);
}

void strtype::show()
{
    cout << p << " - length: " << len;
    cout << "\n";
}

int main()
{
    strtype s1("This is a test."), s2("I like C++.");

    s1.show();
    s2.show();

    return 0;
}
```

3. On your own, rework any program from the preceding chapter so that it uses a reference.